What you're describing in Google Chrome is a mix of several isolation techniques—not just one.

🧩 Short answer

Chrome tabs are sandboxed processes, mainly using:

OS-level process isolation

Sandboxing mechanisms

User namespaces (on Linux)

NOT separate kernels (no full VM per tab)

🔍 How Chrome actually isolates tabs

1. Separate processes (the core idea)

Each tab (more precisely, each site instance) runs in its own process.

This is called site isolation.

👉 If one tab crashes or gets compromised, it doesn't easily affect others.

2. Sandbox (security boundary)

Each renderer process runs inside a sandbox, which:

Restricts file system access

Blocks direct hardware access

Limits system calls

This is the main protection layer.

3. User namespaces (Linux-specific)

On Linux systems, Chrome uses User namespace:

The tab process thinks it has its own users/permissions

But it's actually restricted by the host

This strengthens the sandbox.

4. OS-specific sandboxing

Depending on the OS:

Windows: uses restricted tokens + job objects

macOS: uses Apple's sandbox APIs

Linux: uses namespaces + seccomp-bpf

5. Not a separate kernel ❌

Chrome does NOT run each tab in a separate kernel.

That would require:

Virtual machines (like KVM, Hyper-V)

Much heavier resource usage

Instead, Chrome shares the same kernel but locks down processes tightly.

🧠 Think of it like this

Separate kernel (VM): like running a whole new computer → heavy

Chrome sandbox: like putting each tab in a locked room inside the same building → lightweight but secure

⚠️ Important nuance

Even though it's sandboxed:

Bugs in the browser or kernel can still allow sandbox escape

That’s why Chrome also uses:

Frequent updates

Multi-layer defenses

✅ Final takeaway

Chrome tab isolation is:

👉 Sandboxed processes + OS security features (including user namespaces)

👉 NOT full virtualization or separate kernels

If you want, I can break down how a real exploit tries to escape the Chrome sandbox—that’s where things get really interesting.